TinyML Sound Recognition for Deaf and Hard-of-Hearing Awareness

On-device detection of important environmental sounds such as sirens, dog barking, and clock alarm.

Members: Ethan Jia, Yuming Chen | EE 446 TinyML Final Project

Problem and Motivation

Why TinyML is useful for assistive environmental sound awareness

Problem

Deaf and hard-of-hearing users may miss important sounds in daily environments.

Examples: emergency sirens, dog barking, alarms, and other attention-critical events.

Missing these cues can reduce safety, independence, and situational awareness.

TinyML Opportunity

Run sound recognition directly on a small low-power device.

Avoid continuous cloud streaming of private audio.

Provide fast local alerts through Serial output, vibration, LED, BLE, or mobile/dashboard display.

Success goal: identify key sound classes reliably while fitting memory and latency limits.

Task, Data, and Metrics

Audio classification setup

Task Definition

Input: short audio window from microphone or test audio sample.

Output: predicted environmental sound class.

Target classes: siren, dog bark, crying baby, glass breaking, train, and background (cracking fire).

Application: alert user when safety-relevant sound is detected.

Data Pipeline

Dataset: ESC-50 selected sound classes.

Data collected: 23 min 20 sec total.

Samples: 220 total audio samples across 7 classes.

Class count: 32 samples for most classes; dog has 28 samples.

Preprocessing: resampling, windowing, and MFE feature extraction.

Train / test split: 79% / 21%.

Metrics

Accuracy: 81.67% test accuracy.

Evaluation: confusion matrix and F1 score.

Model size: 57.8 KB.

RAM usage: 41.0 KB.

Latency: 682 ms.

System Overview

End-to-end inference workflow

Hardware Target

Arduino Nano 33 BLE Sense Rev2

Input: On-board microphone

Output: BLE interface

Software Stack

Feature extraction block

TFLite model

Class post-processing and alert threshold

User Interaction

Only important sounds trigger alert

Class label and confidence shown

Future: mobile notification / haptic alert

Baseline ML Models

Offline models before TinyML compression

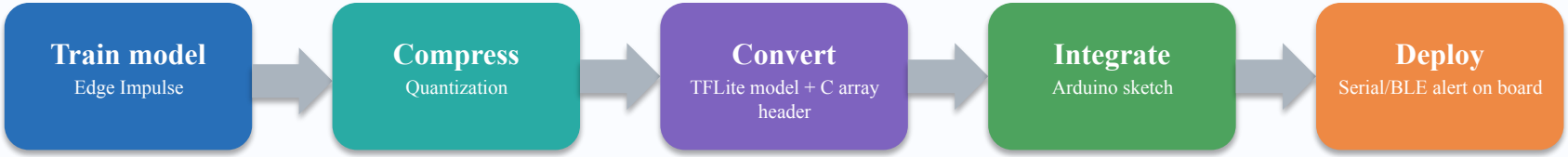
Model Architecture

Input: MFE audio features
Feature shape: 3,960 values per audio window
Reshape into 2D feature map with 36 MFE filters
CNN: Conv2D + pooling layers for time-frequency pattern recognition
Dropout reduces overfitting
Output: 7 environmental sound classes

Model	Input Feature	Train Metric	Test Metric	Notes
Float32 CNN baseline	MFE	89.0%	81.9%	Best offline model
INT8 quantized CNN	MFE	89.0%	81.67%	TinyML-ready
Final deployed model	MFE	N/A	Real-time Device Test	Runs on arduino

TinyML Pipeline

From trained audio model to Arduino deployment



Deployment Details

Model file: exported C++ model array from Edge Impulse / TFLite

Tensor arena: set based on TFLite Micro memory requirement

Operators: Conv2D, Pooling, Reshape

Final model storage: 57.8 KB

Runtime RAM estimate: 41.0 KB

Audio Inference Loop

Collect audio window from microphone.

Extract same features used during training.

Run inference on-device.

Apply threshold and trigger alert for important classes.

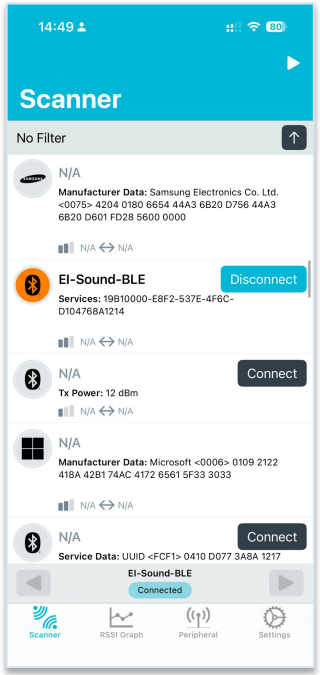
Advanced Component Integration

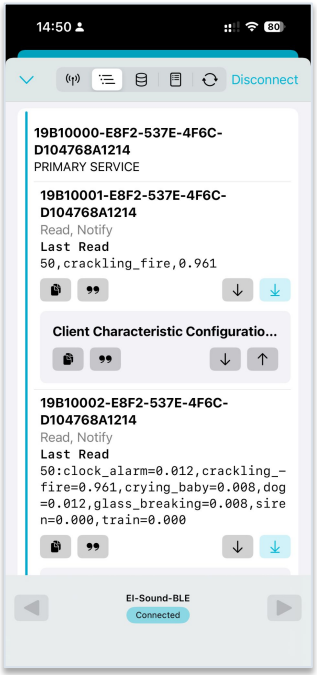
BLE output

This project adds Bluetooth Low Energy communication to an Edge Impulse sound detection model running on the Arduino Nano 33 BLE Sense. Instead of only printing inference results to the Arduino Serial Monitor, the board wirelessly broadcasts the results to a phone or computer.

A BLE client, such as the nRF Connect mobile app, can scan for the device, connect to it, and subscribe to inference result notifications.

This characteristic sends the top classification result. Each notification includes the inference count, predicted class, and confidence score.





Tiny Model Results

Compare baseline and deployable models

Model Type	Train Metric	Test Metric	Size	Latency	RAM	Deployed?
Baseline model	89.0%	81.9%	136.8KB	2430ms	156.4KB	No
Compressed model	89.0%	81.67%	57.8KB	682ms	41.0KB	Yes
Final on-device model	89.0%	81.67%	57.8KB	682ms	41.0KB	Yes

Expected Analysis

INT8 quantization reduced latency from 2430 ms to 682 ms.
Model size decreased from 136.8 KB to 57.8 KB.
Accuracy only dropped from 81.90% to 81.67%.

Per-Class Performance

Strong classes: train, clock alarm, siren, and crackling fire.
Harder classes: dog bark and glass breaking showed more confusion.
Safety-critical sounds such as siren still performed well.

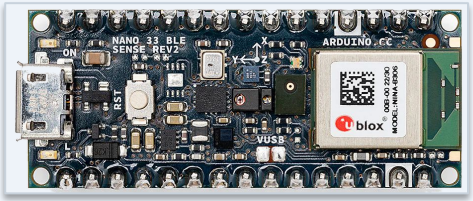
TinyML Constraints

Final model fits the Arduino deployment target.
RAM estimate: 41.0 KB.
Final model size: 57.8 KB.

On-Device Deployment Evidence

Proof that inference runs on hardware

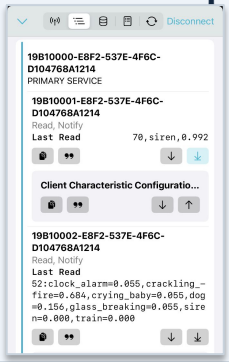
Board / Setup Photo



Serial Monitor Output

```
Starting inferencing in 5 seconds...
Recording...
Recording samples: 0/32000
Recording samples: 16384/32000
Recording done
Timing: DSP 442 ms, inference 482 ms, anomaly 0 ms
#Classification predictions:
    clock_alarm: 0.000000
    crackling_fire: 0.000000
    crying_baby: 0.000000
    dog: 0.000000
    glass_breaking: 0.000000
    siren: 0.000000
    train: 0.996094
```

BLE Alert Output



Evidence Checklist

Serial Monitor shows real-time class probabilities.

BLE output sends detected class and confidence.

Example result: train detected with 99.6% confidence.

Final model uses INT8 quantized CNN from the results slide.

Key Challenges and Solutions

Technical issues and how they were addressed

Challenge	Why It Matters	Solution / Current Status
Model too large for Flash/RAM	Audio models can exceed MCU memory	Use smaller architecture, INT8 quantization, remove unused ops
Noisy real-world audio	Class predictions may be unstable	Add background noise augmentation and confidence threshold
Class imbalance / limited samples	Safety classes may have few examples	Collect more samples or use public dataset + targeted custom recordings
On-board input mismatch	Training features must match Arduino features	Keep sampling rate, window size, and preprocessing consistent
Demo reliability	Live audio demos may be inconsistent	Prepare backup video, logs, screenshots, and known test samples

Conclusion and Takeaways

What worked, what was learned, and next steps

What Worked

Built an audio classification pipeline for assistive sound awareness.

Converted model toward TinyML deployment.

Demonstrated on-device or hardware-realistic inference evidence.

What We Learned

TinyML requires balancing accuracy, model size, memory, and latency.

Audio preprocessing consistency is critical.

Safety-relevant classes need robust testing under noise.

Future Improvements

More real-world samples and noise conditions.

Haptic notification for user-facing alert.

Personalized fine-tuning for user environment.

Lower-power continuous listening optimization.

References

Datasets, tools, papers, and reused code

Reference List

Dataset: ESC-50 https://github.com/karolpiczak/ESC-50

Tools: TensorFlow Lite/ Edge Impulse / Arduino IDE

Hardware: Arduino Nano 33 BLE Sense Rev2

EE 446 Final Project Presentation and Demo Guidelines / Rubric.

This reference slide does not count toward the 10 content-slide limit.